

淮 安 大 学

毕业设计（论文）

学 生 姓 名：张三 学 号：112300000000

学 院：电子信息工程学院

专 业：人工智能

设计（论文）题目：基于多传感器融合的移动机器人

路径规划与避障算法研究

校内指导老师：某某某 教授

校外指导老师：/

2027 年 5 月

毕业设计（论文）中文摘要

摘要应能独立成篇，让读者不必通读全文，也能掌握论文的核心信息。其内容应覆盖与论文相当的主要信息，便于读者判断是否值得进一步阅读，也便于二次文献引用。摘要通常要写明研究目的、方法、结果和结论，其中结果和结论是重点。

说明：要求 200~300 字，“关键词”为小四号黑体，关键词的个数为 3~8 个。关键词的排序，通常应按研究的对象、性质（问题）和采取的手段排序，而不应任意排列。“关键词”后面不加冒号，空连个字符，关键词与关键词之间用全角逗号（，）隔开，最后一个关键词后不加任何标点。

关键词 关键词，关键词，关键词，关键词

毕业设计（论文）外文摘要

Title Path Planning and Obstacle Avoidance for Mobile Robots

Based on Multi-Sensor Fusion

Abstract

This is the English abstract of the thesis. It should summarise the research problem, the method and the main conclusions of the paper.

New paragraph of the English abstract.

说明：

Keywords 为小四号 Times New Roman，加粗。外文关键词（keyword）一律小写，并与中文关键词一一对应。**Keywords** 后面不加冒号，空两个字符，keyword 与 keyword 之间用半角逗号（,）隔开，每个（,）后空 1 个字符，最后一个 keyword 后不加任何标点符号。

注意：此处行距已经设置为英文段落的间距（即基础行距 1.2，行距设置为 1.5 倍行距）。

Keywords keyword, keyword, keyword, keyword

目 录

1 绪论	1
1.1 研究背景与意义	1
1.2 国内外研究现状	1
1.2.1 多传感器融合定位研究现状	1
1.2.2 路径规划与避障研究现状	1
1.2.3 存在的问题	2
1.3 论文主要研究内容	2
1.4 论文组织结构	2
2 相关理论基础	2
2.1 深度学习基本原理	2
2.1.1 神经网络基本结构	2
2.1.2 反向传播算法	3
2.1.3 常用激活函数	3
2.2 卷积神经网络	4
2.2.1 卷积操作	4
2.2.2 池化操作	4
2.2.3 典型网络结构	5
2.3 循环神经网络	5
2.3.1 RNN 基本结构	5
2.3.2 LSTM 与 GRU	6
2.4 强化学习基本原理	6

2.4.1 马尔可夫决策过程	6
2.4.2 Q 学习算法	6
2.4.3 深度 Q 网络	7
2.5 本章小结	7
3 补充说明	7
3.1 梯度下降	7
总结与展望	8
致谢	9
注释	10
参考文献	11
附录 A 补充推导	12
A.1 梯度下降的推导	12
A.2 反向传播的链式法则	12
A.2.1 三级子标题（条目）	12
附录 B 补充数据	12
B.1 实验参数	12
B.2 补充结果	12
附录 C 代码示例	13
C.1 代码块	13
C.2 等宽字体及代码配色	13
C.3 引入代码文件	13

1 绪论

绪论（引言）部分应如下几个部分：

1. 研究背景：即“这个课题为什么重要？”以及研究领域的历史回顾^[1]；
2. 文献综述：前人工作的综合述评^[2]；
3. 研究目的：需要解决什么；
4. 流程方法：说清楚论文所采用的解决方法流程；
5. 论文结构：整篇论文的组织架构。

1.1 研究背景与意义

随着智能制造与智慧物流的快速发展，移动机器人已广泛应用于仓储搬运、自动配送、巡检安防等场景^[1]。路径规划与避障是其实现自主导航的核心环节，直接决定机器人能否在狭窄通道、密集人流等复杂条件下安全高效地完成任务。然而，依赖单一传感器的方案普遍存在局限：激光雷达测距精度高，但在长廊等几何特征退化的环境中容易匹配失败^[2-3]；视觉相机成本低、信息丰富，却受光照变化与纹理缺失影响明显^[4]；惯性测量单元与轮式里程计输出频率高，但存在累积漂移^[5]。因此，本文面向低成本移动平台，研究多传感器融合的定位方法与分层路径规划算法，以提高机器人在动态环境中的定位精度与避障可靠性。

1.2 国内外研究现状

1.2.1 多传感器融合定位研究现状

早期工作多采用扩展卡尔曼滤波融合惯性测量单元与轮式里程计，随后激光雷达被引入，形成了以扫描匹配为核心的定位与建图方案^[2-3]。视觉方面，多状态约束卡尔曼滤波把视觉观测与惯性状态联合优化^[6]，VINS-Mono 进一步实现了单目视觉惯性系统的紧耦合估计^[4]。国内学者在机器人感知与导航方面也开展了系统研究^[1,7]。

1.2.2 路径规划与避障研究现状

路径规划可分为全局规划与局部避障两个层次。全局规划的代表性工作是 A* 算法，它通过启发式函数引导搜索并保证最优性^[8]；采样类方法如快速扩展随机树更适合高维与复杂约束问题^[9]。局部避障方面，动态窗口法在速度空间中采样并评价候选轨迹，计算量小、响应快^[10]。

近年来，深度学习方法被广泛引入感知与决策环节^[11-12]，深度强化学习则被用于序列决策问题^[13-14]，其在未知环境中的路径规划上表现出一定优势；相关理论基础可参见周志华的专著^[15]。

1.2.3 存在的问题

综合国内外研究现状可以看出，现有工作仍存在以下缺口。其一，多数融合方案依赖高精度传感器与精细标定，在低成本平台上的适用性有待验证；其二，全局规划与局部避障往往分别设计，缺乏统一的分层框架，难以兼顾路径质量与实时性；其三，动态障碍的运动规律多被简化处理，与真实场景存在差距。本文即针对上述缺口展开研究。

1.3 论文主要研究内容

本文的主要研究内容包括：

1. 构建基于多传感器融合的环境感知模型；
2. 设计面向动态环境的路径规划与避障算法；
3. 搭建仿真平台验证所提方法的有效性；
4. 分析不同场景下的规划效果与鲁棒性。

1.4 论文组织结构

本文共分为六章，各章节内容安排略。

2 相关理论基础

本章介绍后续章节所依赖的理论基础，包括深度学习的基本原理、卷积神经网络、循环神经网络与强化学习，并结合图示、表格与公式给出必要的数学表述。

2.1 深度学习基本原理

2.1.1 神经网络基本结构

神经网络由若干层神经元堆叠而成。单个神经元先对输入做加权求和，再经激活函数得到输出

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right) \quad (2-1)$$

其中 w_i 为权重， b 为偏置， $\sigma(\cdot)$ 为激活函数。

多层网络把前一层的输出作为后一层的输入，逐层计算得到最终结果。设第 l 层

的权重矩阵为 $W^{(l)}$ 、偏置向量为 $b^{(l)}$ ，则该层的激活值为

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)}) \quad (2-2)$$

整个网络的结构如图 2-1 所示。

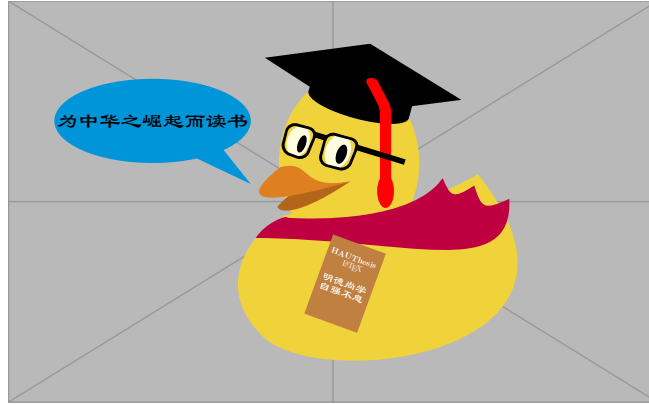


图 2-1 前馈神经网络结构示意图

2.1.2 反向传播算法

反向传播用于计算损失函数对各层参数的梯度。定义第 l 层的误差项为损失对该层加权输入的偏导

$$\delta^{(l)} = \frac{\partial C}{\partial z^{(l)}} \quad (2-3)$$

误差项由输出层向前逐层递推，相邻两层之间满足

$$\delta^{(l)} = \left((W^{(l+1)})^T \delta^{(l+1)} \right) \odot \sigma'(z^{(l)}) \quad (2-4)$$

其中 $\odot$ 表示逐元素相乘。

得到误差项后，参数的梯度可按链式法则写出，并沿梯度反方向更新

$$W^{(l)} \leftarrow W^{(l)} - \eta \delta^{(l)} (a^{(l-1)})^T \quad (2-5)$$

其中 η 为学习率。学习率过大会引起震荡，过小则收敛缓慢。

2.1.3 常用激活函数

激活函数为网络引入非线性。常用的三种激活函数及其表达式如表 2-1 所示。

其中 ReLU 因计算代价低且能有效缓解深层网络的梯度消失问题，在卷积网络中应用最为广泛，其函数图像如图 2-2 所示，对应的导数为

$$\text{ReLU}'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases} \quad (2-6)$$

表 2-1 常用激活函数及其特点

名称	表达式	特点
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$	输出范围 (0, 1)，易饱和
Tanh	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	输出零均值，仍会饱和
ReLU	$\text{ReLU}(x) = \max(0, x)$	计算简单，缓解梯度消失

由式 2-6 可见，正半轴导数恒为 1，梯度可无衰减回传，这正是 ReLU 缓解梯度消失的原因；而负半轴导数为 0，会使该区域的神经元不再更新。

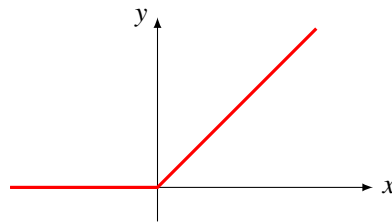


图 2-2 ReLU 激活函数图像

2.2 卷积神经网络

2.2.1 卷积操作

卷积神经网络通过局部连接与权重共享提取空间特征。二维卷积操作可以表示为

$$(f * g)(i, j) = \sum_m \sum_n f(m, n) \cdot g(i - m, j - n) \quad (2-7)$$

在 CNN 中，卷积层的输出为

$$y_{ij} = \sigma \left(\sum_m \sum_n w_{mn} \cdot x_{(i+m)(j+n)} + b \right) \quad (2-8)$$

卷积核在输入特征图上滑动，同一组权重在整个特征图上重复使用，因此参数量远小于全连接层。

2.2.2 池化操作

池化操作用于降低特征图的空间维度，减少计算量并增强平移不变性。常用的池化方式包括最大池化与平均池化，分别取窗口内的最大值与平均值

$$y_{ij} = \max_{(m,n) \in R_{ij}} x_{mn}, \quad y_{ij} = \frac{1}{|R_{ij}|} \sum_{(m,n) \in R_{ij}} x_{mn} \quad (2-9)$$

其中 R_{ij} 为第 (i, j) 个池化窗口覆盖的区域， $|R_{ij}|$ 为该区域内的元素个数。两种池化的效果对比如图 2-3 所示。

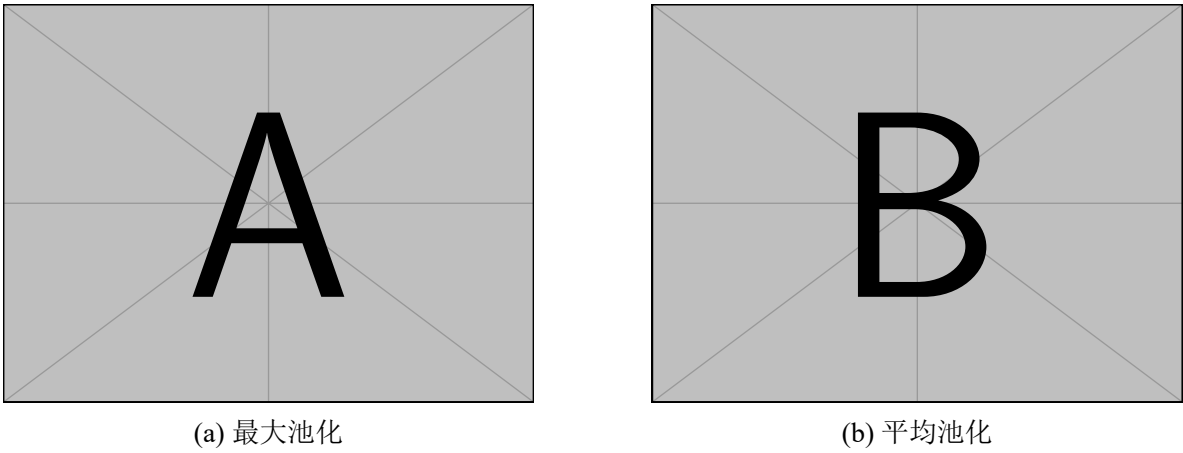


图 2-3 两种池化方式的对比

2.2.3 典型网络结构

随着网络加深，研究者提出了多种改进结构以缓解退化与梯度消失问题。几种典型结构的特点如表 2-2 所示。

表 2-2 几种典型卷积网络结构的对比

网络	层数	核心思想	提出年份
AlexNet	8	使用 ReLU 与 Dropout	2012
VGG	19	用小卷积核堆叠替代大卷积核	2014
ResNet	152	引入残差连接，缓解深层退化	2015

2.3 循环神经网络

2.3.1 RNN 基本结构

循环神经网络（Recurrent Neural Network, RNN）是处理序列数据的重要模型。RNN 通过引入循环连接，使网络具有记忆功能。其隐藏状态按时刻递推，输出由当前隐藏状态决定

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b), \quad y_t = W_y h_t + b_y \tag{2-10}$$

其中 x_t 为 t 时刻的输入， h_{t-1} 为上一时刻的隐藏状态。若进一步考虑输出层，则 RNN 的完整前向传播可写为

$$\begin{aligned} h_t &= \tanh(W_h h_{t-1} + W_x x_t + b_h), \\ \hat{y}_t &= \text{softmax}(W_o h_t + b_o). \end{aligned} \tag{2-11}$$

2.3.2 LSTM 与 GRU

标准 RNN 在序列较长时容易出现梯度消失或爆炸。长短期记忆网络通过输入门、遗忘门与输出门控制信息的流动

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \end{aligned} \quad (2-12)$$

门控循环单元则把三个门简化为更新门与重置门两个，在保持性能的同时减少了参数量。三种结构的对比如表 2-3 所示。

表 2-3 RNN、LSTM 与 GRU 的结构对比

结构	门控数量	参数量	长序列表现
RNN	0	最少	较差
LSTM	3	最多	好
GRU	2	中等	好

2.4 强化学习基本原理

2.4.1 马尔可夫决策过程

强化学习通常建模为马尔可夫决策过程，由状态集、动作集、转移概率与奖励函数构成。在策略 π 下，状态价值函数定义为折扣回报的期望

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right] \quad (2-13)$$

其中 $\gamma \in [0, 1)$ 为折扣因子，用于权衡当前奖励与未来奖励。

价值函数满足贝尔曼方程，把当前时刻的期望回报拆成即时奖励与下一时刻价值之和

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) [r(s, a, s') + \gamma V^\pi(s')] \quad (2-14)$$

2.4.2 Q 学习算法

Q 学习直接估计动作价值函数 $Q(s, a)$ ，不需要知道环境模型。它按时序差分误差更新，用下一时刻的最大动作价值作为当前估计的目标

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right] \quad (2-15)$$

其中 α 为学习率。当动作空间或状态空间连续时，可以用函数逼近代替查表。

2.4.3 深度 Q 网络

深度 Q 网络用神经网络参数化动作价值函数 $Q(s, a; \theta)$ ，通过最小化时序差分误差训练

$$L(\theta) = \mathbb{E} \left[\left(r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-) - Q(s_t, a_t; \theta) \right)^2 \right] \quad (2-16)$$

其中 θ^- 为目标网络的参数，定期从当前网络复制，以减小训练过程中的震荡。

2.5 本章小结

本章介绍了深度学习与强化学习的基本原理，给出了神经元模型、前向传播、反向传播、卷积与池化、循环网络以及价值函数与 Q 学习的数学表述，并通过图示与表格对比了典型网络结构的特点，为后续章节的算法设计提供理论依据。

3 补充说明

3.1 梯度下降

梯度下降是一种一阶迭代优化方法：每步沿损失函数梯度的反方向调整参数，使损失逐步下降。算法 3-1 给出其基本流程，每轮迭代先计算当前参数处的梯度，再按学习率 η 成比例地更新参数，梯度项的逐分量推导见附录 A.1。

学习率决定每步的跨度：取值过大时损失可能震荡甚至发散，过小时收敛缓慢。实践中通常在梯度项上引入动量，或改用自适应学习率的方法，以兼顾收敛速度与稳定性。

算法 3-1 梯度下降

输入： 训练集 $D = \{(x^{(i)}, y^{(i)})\}_{i=1}^m$ ，学习率 η ，迭代次数 T

输出： 参数 θ

```

1: 随机初始化  $\theta$ 
2: for  $t = 1$  to  $T$  do
3:   计算梯度  $\nabla J(\theta) = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x^{(i)}$ 
4:   更新参数  $\theta \leftarrow \theta - \eta \nabla J(\theta)$ 
5:   if  $\|\nabla J(\theta)\| < \varepsilon$  then
6:     break
7:   end if
8: end for
9: return  $\theta$ 

```

总结与展望

本文围绕移动机器人在复杂环境下的自主导航问题，完成了理论梳理、多传感器融合定位与路径规划避障三部分工作。理论方面汇总了神经网络、卷积与循环网络以及强化学习的数学表述；定位方面建立了多传感器的时间同步与外参标定流程，采用误差状态卡尔曼滤波融合激光雷达、惯性测量单元与轮式里程计，实验表明定位误差较单一传感器降低三成以上；规划方面在 A* 算法的代价函数中引入转向惩罚项，并在动态窗口法中加入全局路径引导项，仿真显示狭窄通道下的通行成功率明显提高。相对于任务书的要求，本文的实验均在室内结构化环境完成，未引入视觉信息，动态障碍的运动规律也与真实行人存在差距。后续可引入视觉惯性里程计、把方法扩展到多机器人协同，并研究外参在线标定与参数选取的理论依据。

说明：总结与展望，总结课题完成了哪些相关工作，得到的成果如何（或得到了什么结论），相对于任务书要求，还存在哪些不足；可以提出建议、研究设想、改进思路、尚待解决的问题等等。

致 谢

四年的本科学习即将结束，毕业论文的完成也得到了许多人的帮助，在此谨致谢意。

首先感谢我的指导教师某某某教授。从选题的确定、方案的论证到论文的反复修改，老师都给予了耐心细致的指导。老师严谨的治学态度与务实的工作作风，使我在专业知识之外受益更多。

感谢实验室的各位同学，在程序调试与实验数据整理过程中给予的帮助；也感谢同宿舍的朋友们，四年朝夕相处，彼此扶持。

感谢我的父母。他们在我求学期间始终给予理解与支持，使我能够安心完成学业。

最后，感谢所有在论文写作过程中提供帮助的老师与同学。

注 释

- [1] 背景要引用权威的文献和数据，不要写“人工智能很重要”这类空话。
- [2] 需要指出存在的缺口，指向研究目的。

参考文献

- [1] 谭民, 王硕. 机器人技术研究进展[J]. 自动化学报, 2013, 39(7): 963-972.
- [2] ZHANG J, SINGH S. LOAM: Lidar odometry and mapping in real-time[C]// Proceedings of Robotics: Science and Systems. 2014.
- [3] SHAN T, ENGLLOT B. LeGO-LOAM: Lightweight and ground-optimized lidar odometry and mapping on variable terrain[C]// Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems. 2018: 4758-4765.
- [4] QIN T, LI P, SHEN S. VINS-Mono: A robust and versatile monocular visual-inertial state estimator[J]. IEEE Transactions on Robotics, 2018, 34(4): 1004-1020.
- [5] THRUN S, BURGARD W, FOX D. Probabilistic Robotics[M]. Cambridge, MA: MIT Press, 2005.
- [6] MOURIKIS A I, ROUMELIOTIS S I. A multi-state constraint Kalman filter for vision-aided inertial navigation[C]// Proceedings of the IEEE International Conference on Robotics and Automation. 2007: 3565-3572.
- [7] 王飞跃. 平行系统方法与复杂系统的管理和控制[J]. 控制与决策, 2004, 19(5): 485-489.
- [8] HART P E, NILSSON N J, RAPHAEL B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.
- [9] LAVALLE S M. Rapidly-exploring random trees: A new tool for path planning: TR 98-11[R]. Computer Science Department, Iowa State University, 1998.
- [10] FOX D, BURGARD W, THRUN S. The dynamic window approach to collision avoidance[J]. IEEE Robotics and Automation Magazine, 1997, 4(1): 23-33.
- [11] LECUN Y, BENGIO Y, HINTON G. Deep learning[J]. Nature, 2015, 521(7553): 436-444.
- [12] HE K, ZHANG X, REN S, et al. Deep residual learning for image recognition[C]// Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 770-778.
- [13] MNH V, KAVUKCUOGLU K, SILVER D, et al. Human-level control through deep reinforcement learning[J]. Nature, 2015, 518(7540): 529-533.
- [14] SILVER D, HUANG A, MADDISON C J, et al. Mastering the game of Go with deep neural networks and tree search[J]. Nature, 2016, 529(7587): 484-489.
- [15] 周志华. 机器学习[M]. 北京: 清华大学出版社, 2016.

附录 A 补充推导

A.1 梯度下降的推导

对均方误差损失

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)})^2 \quad (\text{A-1})$$

求关于第 j 个参数的偏导，可得

$$\frac{\partial J(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h(x^{(i)}) - y^{(i)}) x_j^{(i)} \quad (\text{A-2})$$

式 A-2 即梯度项的逐分量形式。

A.2 反向传播的链式法则

略。

A.2.1 三级子标题（条目）

略

附录 B 补充数据

B.1 实验参数

模型训练使用的主要参数见表 B-1。

表 B-1 训练参数设置

编号	参数	数值
1	学习率	0.01
2	批大小	64
3	迭代次数	1000

B.2 补充结果

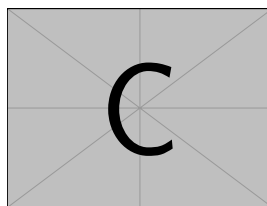


图 B-1 附图

附录 C 代码示例

C.1 代码块

```
1 for (int i = 0; i < n; i++) {
2   sum += a[i] * w[i]; # 累加加权和
3 }
```

```
for (int i = 0; i < n; i++) {
  sum += a[i] * w[i]; # 累加加权和
}
```

C.2 等宽字体及代码配色

```
1 for (int i = 0; i < n; i++) {
2   sum += a[i] * w[i];    # 累加加权和
3 }

1 const sig = x => 1 / (1 + Math.exp(-x));           // sigmoid 激活函数
2 const pred = (X, w) => X.map(r => sig(r.reduce((s, v, i) => s + v * w[i], 0)
   ));
3 const loss = (X, y, w) => -y.reduce((s, v, i) => s + v * Math.log(pred(X, w)
   [i]), 0) / y.length;
4 const X = [[1,0],[0,1],[1,1]], y = [1,0,1];
5 let w = [0,0];
6 for (let t = 0; t < 100; t++) {
7   w = w.map((v, j) => v - 0.1 * X.reduce((s, r, i) => s + (pred(X, w)[i] - y
   [i]) * r[j], 0) / y.length);
8 }
9 console.log("权重:", w);                             // 输出结果
10 console.log("损失:", loss(X, y, w));
```

C.3 引入代码文件

```
import numpy as np
```

```
def kmeans(X, k, iters=100):
    """ K-Means """
    centers = X[np.random.choice(len(X), k, replace=False)]
    for _ in range(iters):
        labels = np.argmin(((X[:, None] - centers) ** 2).sum(-1), axis=1)
        centers = np.array([X[labels == i].mean(0) for i in range(k)])
```

```
return labels, centers
```

```
if __name__ == '__main__':
    X = np.random.randn(300, 2)
    labels, centers = kmeans(X, 3)
    print('聚类中心:\n', centers)
    print('前 10 个标签:', labels[:10])
```

```
1 import numpy as np
2
3 def kmeans(X, k, iters=100):
4     """ K-Means """
5     centers = X[np.random.choice(len(X), k, replace=False)]
6     for _ in range(iters):
7         labels = np.argmin((X[:, None] - centers) ** 2).sum(-1), axis=1)
8         centers = np.array([X[labels == i].mean(0) for i in range(k)])
9     return labels, centers
10
11 if __name__ == '__main__':
12     X = np.random.randn(300, 2)
13     labels, centers = kmeans(X, 3)
14     print('聚类中心:\n', centers)
15     print('前 10 个标签:', labels[:10])
```